

第四章 第三节 Python进阶篇——类的其它属性

- 方法重写与多态性

示例：多态性的使用

多态性是指允许不同类的对象对同一消息做出响应的能力，但响应的方式取决于对象的实际类型。

```
class Animal:
    def speak(self):
        raise NotImplementedError("Subclasses must implement this method")

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

def animal_sound(animal):
    print(animal.speak())

# 创建 Animal 类的实例
dog = Dog()
cat = Cat()

animal_sound(dog) # 输出: Woof!
animal_sound(cat) # 输出: Meow!
```

第四章 第三节 Python进阶篇——类的其它属性

- 方法重写与多态性

示例：多态性的使用

在这个示例中，Dog 和 Cat 类都实现了 `speak` 方法，展示了多态性。

第四章 第三节 Python进阶篇——类的其它属性

- 类的私有属性和方法

在Python中，类的私有属性和方法允许我们封装数据和行为，限制对类成员的直接访问。

- 私有化的限制

Python并没有严格意义上的私有化，但约定俗成的命名方式可以模拟私有化的效果。

示例：模拟私有属性

```
class Account:
```

```
    def __init__(self, balance):
        self.__balance = balance # 模拟私有属性

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

```
# 创建 Account 类的实例
```

```
my_account = Account(1000)
```

```
print(my_account.get_balance()) # 输出: 1000
```

```
my_account.deposit(500)
```

```
print(my_account.get_balance()) # 输出: 1500
```

```
# print(my_account.__balance) # 这将引发错误，因为 __balance 是"私有"的
```

第四章 第三节 Python进阶篇——类的其它属性

- 方法重写与多态性

使用__private和_protected属性

在Python中，使用双下划线__前缀的属性或方法名会在类外部被转换成单下划线_前缀，这是一种名称改编（name mangling）机制，以避免与子类中的属性冲突。

示例：使用__private属性

```
class Person:
```

```
    def __init__(self, name):
```

```
        self.__name = name # 私有属性
```

```
    def get_name(self):
```

```
        return self.__name
```

```
# 创建 Person 类的实例
```

```
person = Person("Alice")
```

```
print(person.get_name()) # 输出: Alice
```

```
# print(person.__name) # 这将引发错误，因为 __name 是私有的
```

第四章 第三节 Python进阶篇——类的其它属性

- 方法重写与多态性

示例：使用 `_protected` 属性

```
class Employee(Person):
```

```
    def set_name(self, new_name):
```

```
        self.__name = new_name # 受保护的属性，可以在子类中访问
```

```
employee = Employee("Bob")
```

```
employee.set_name("Charlie")
```

```
print(employee.get_name()) # 输出: Charlie
```

第四章 第三节 Python进阶篇——类的其它属性

- 静态方法和类方法

在Python中，静态方法和类方法是与类相关联的函数，但它们在访问和行为上有所不同。

静态方法(`@staticmethod`)的使用

静态方法是与类相关但不需要类或实例数据的方法。它们通过`@staticmethod`装饰器定义。

示例：静态方法

python复制代码class `MathUtils`:

```
    @staticmethod
```

```
    def add(a, b):
```

```
        return a + b
```

使用静态方法

```
print(MathUtils.add(3, 4)) # 输出: 7
```

静态方法可以被类和类的实例直接调用，而不需要类或实例的引用。

第四章 第三节 Python进阶篇——类的其它属性

- 静态方法和类方法

在Python中，静态方法和类方法是与类相关联的函数，但它们在访问和行为上有所不同。

类方法(`@classmethod`)的应用

类方法是与类相关的方法，它们的第一个参数是类本身，通常命名为cls。

示例：类方法

```
class Animal:
```

```
    count = 0 # 类变量
```

```
    @classmethod
```

```
    def increment_count(cls):
```

```
        cls.count += 1
```

```
    @classmethod
```

```
    def get_count(cls):
```

```
        return cls.count
```

```
# 使用类方法
```

```
Animal.increment_count()
```

```
print(Animal.get_count()) # 输出: 1
```

类方法通常用于访问或修改类状态。

第四章 第三节 Python进阶篇——类的其它属性

- 静态方法和类方法的区别

静态方法不需要类或实例的引用。

类方法需要类的引用，并且可以访问和修改类变量。

```
class MyClass:
```

```
    @staticmethod
```

```
    def my_static_method():
```

```
        print("This is a static method.")
```

```
    @classmethod
```

```
    def my_class_method(cls):
```

```
        print("This is a class method of", cls)
```

```
# 调用静态方法
```

```
MyClass.my_static_method()
```

```
# 调用类方法
```

```
MyClass.my_class_method()
```


第四章 第三节 Python进阶篇——类的其它属性

- 属性装饰器

属性装饰器是Python中一个强大的特性，它允许我们控制对属性的访问和赋值。

使用@**property**创建访问器

@**property**装饰器可以将一个方法转变为属性访问器，允许我们读取属性值。

示例：使用@**property**

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    def set_radius(self, value):
        if value < 0:
            raise ValueError("Radius cannot be negative")
        self._radius = value

# 创建Circle类的实例
circle = Circle(10)
print(circle.radius) # 输出: 10
circle.set_radius(20)
print(circle.radius) # 输出: 20
```

第四章 第三节 Python进阶篇——类的其它属性

- 属性装饰器

使用@装饰器管理属性访问

除了@property，我们还可以定义自己的装饰器来管理属性的访问。

示例：自定义属性装饰器

```
def readonly_property(func):  
    def wrapper(self):  
        raise AttributeError("This attribute is read-only")  
    return property(wrapper)
```

```
class ImmutableCircle(Circle):
```

```
    @readonly_property
```

```
    def radius(self):
```

```
        pass
```

```
# 创建ImmutableCircle类的实例
```

```
immutable_circle = ImmutableCircle(10)
```

```
print(immutable_circle.radius) # 输出: 10
```

```
# immutable_circle.set_radius(20) # 将引发错误，因为radius是只读的
```

第四章 第三节 Python进阶篇——类的其它属性

- 属性装饰器

属性装饰器的高级用法

属性装饰器不仅可以用于封装属性访问，还可以用于实现更复杂的逻辑。

示例：属性装饰器的高级用法

```
def cached_property(func):
```

```
    cache_attr = '_cache_' + func.__name__
```

```
    property_attr = '_property_' + func.__name__
```

```
    @property
```

```
    def wrapper(self):
```

```
        cache_value = getattr(self, cache_attr, None)
```

```
        if cache_value is None:
```

```
            cache_value = func(self)
```

```
            setattr(self, cache_attr, cache_value)
```

```
        return cache_value
```

```
    return wrapper
```

第四章 第三节 Python进阶篇——类的其它属性

- 属性装饰器

```
class ExpensiveComputation:
```

```
    def __init__(self, value):
```

```
        self._value = value
```

```
    @cached_property
```

```
    def result(self):
```

```
        print("Computing expensive result...")
```

```
        return self._value ** 2
```

```
# 创建ExpensiveComputation类的实例
```

```
expensive = ExpensiveComputation(10)
```

```
print(expensive.result) # 输出: Computing expensive result..., 100
```

```
print(expensive.result) # 输出: 100, 但不会再次计算
```